

**AI AND IOT-ENABLED SMART OFFICE SYSTEM
WITH CLOUD-BASED ENERGY OPTIMIZATION**

25-26J-34

Performance Testing Report

Checklist 2

M.A.H.S.THATHSARANI

IT22564672

Supervisor: Prof. Anuradha Jayakody

B.Sc. (Hons) in Information Technology

Specializing in Computer Systems and Network Engineering

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

30/04/2026

1. Introduction and Objective

By dynamically allocating tasks to workers based on their current stress levels, the Smart Office-Stress-Based Task Allocation System is a cutting-edge technology created to maximize workplace efficiency. In order to minimize burnout, the system uses computer vision and machine learning (ML) to detect stress signs in facial expressions and adjust workload accordingly.

This performance testing phase's main goal is to make sure the system can manage many user requests with low latency, especially while processing biometric data and real-time stress detection. Because sluggish ML inference or delayed job allocation might result in an out-of-date representation of an employee's state, which would reduce system efficacy, performance testing is essential for this system.

2. Performance Testing Environment Setup

A regulated testing environment was created to guarantee accuracy and repeatability. To assess the integrated components' baseline performance free from outside network interference, the tests were carried out in a controlled local setting.

Hardware Configuration

A typical development computer with an Intel Core i5 CPU and 8GB of RAM was used as the server environment. In the client-side testing, the Next.js frontend was accessed by browser-based users.

Software Configuration

Windows 10/11 served as the operating system. Next.js and Node.js were used to create the frontend, while Python 3.x and FastAPI were used to develop the backend. PostgreSQL was used to control data durability.

Network Configuration

Testing was conducted over a localhost loopback network (127.0.0.1), providing near-zero latency and allowing accurate measurement of internal processing performance.

3. Tools and Frameworks Used

System performance was assessed at several tiers using a variety of techniques and frameworks. Response time and throughput were measured while simulating concurrent API queries using Apache JMeter. Endpoint validation and functional API testing were conducted using Postman. Network activity and frontend rendering speed were examined using Browser Developer Tools. Request processing efficiency and backend execution time were tracked using FastAPI logs.

4. Testing Methodology and Execution

A progressive approach was taken in the testing technique, which began with a small user base and progressively increased to high-load situations.

Ten concurrent users were first simulated in order to determine baseline performance. After that, the load was increased to 50 users, and subsequently to 100 people in order to see how the system behaved under pressure. Authentication (/auth/login), stress detection (/ml/stress-detect), and task assignment (/tasks/assign) were the main endpoints that were examined.

Endurance testing was carried out by keeping a steady load over time to find any stability concerns, load testing was done to assess typical system performance, and stress testing was used to detect system limitations.

5. Metrics Measured

The following metrics were used to evaluate system performance:

- Response Time (ms): Time taken to process a request and return a response
- Throughput (requests/sec): Number of requests handled per second
- CPU Usage: Processor utilization during peak loads
- Memory Usage: RAM consumption of the system
- Error Rate (%): Percentage of failed or unsuccessful requests

6. Results and Analysis

The system performed efficiently under low to moderate loads but showed increased latency under high concurrent usage.

User Count	Avg. Response Time (ms)	Throughput (req/sec)	Error Rate (%)	CPU Usage (%)
10	120 ms	45	0%	15%
50	450 ms	110	0.5%	65%
100	1200 ms	140	3.2%	92%

The primary bottleneck identified was the ML processing time during stress detection. As concurrent requests increased, CPU usage rose significantly due to model inference operations. Database performance remained stable, although minor delays were observed during peak usage.

The integrated stress detection model achieved an approximate accuracy of 90% under standard testing conditions, contributing to reliable task allocation decisions.

7. Performance Optimization and Recommendations

Several optimizations were identified to improve system performance:

- Implement caching mechanisms (e.g., Redis) to reduce repeated database queries
- Apply database indexing to improve query performance
- Use asynchronous processing for non-critical tasks
- Deploy load balancing in production environments to distribute workload

8. Challenges and Solutions

One major challenge was handling the latency introduced by the ML model during concurrent requests. This was mitigated by optimizing image preprocessing and ensuring the model is loaded only once in memory.

Another challenge was maintaining WebSocket stability under heavy load. This was addressed by implementing connection management and heartbeat mechanisms to prevent stale connections.

9. Ethical and Compliance Considerations

Face photos are among the sensitive biometric data that the system handles. The system uses a privacy-by-design methodology to guarantee both compliance and privacy. Images are processed in memory rather than being permanently saved. Before using the camera, the user must give their consent. Secure communication methods (HTTPS/WSS) are advised for implementation.

10. References

S. Li and W. Deng, “Deep Facial Expression Recognition: A Survey,” *IEEE Transactions on Affective Computing*, 2022.

A. Mollahosseini et al., “AffectNet: A Database for Facial Expression, Valence, and Arousal Computing in the Wild,” *IEEE Transactions on Affective Computing*, 2019.

S. Minaee et al., “Deep-Emotion: Facial Expression Recognition Using Attentional Convolutional Network,” *Sensors*, 2021.

A. Bhattacharyya et al., “Real-time Stress Detection using Facial Emotion Recognition,” *International Journal of Computer Applications*, 2024.